Lab 2: Fine-tune vs prompt

Does fine-tuning beat your best prompt?

Aarne Talman

5 November 2026

LDA-T318 | LLMs and Generative AI for NLP | University of Helsinki

The question

Lab 2

For *your* task, does fine-tuning beat your best prompt?

You compare two ways of getting a model to do your task:

- **prompting**: give the model instructions and, optionally, a few examples, without changing it
- **fine-tuning**: train the model on your labelled examples

Both are scored on the same test examples, which were not used for anything else.

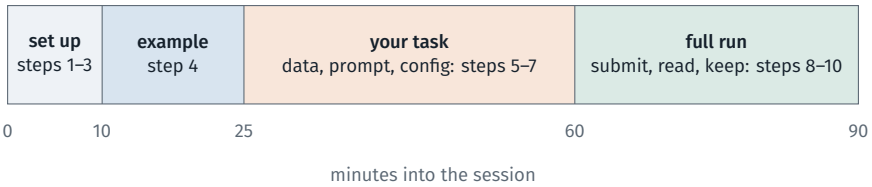
For the final report

- one results table (`summarize_results.py` prints it)
- one loss plot
- one paragraph: was fine-tuning worth the extra time and GPU use?

Minimum: README steps 1–10 once, with default settings. Ideas for going further are in the README.

Everything you need is in `Week-2/Lab-2/README.md`, step by step.

Today



The steps are those in the README. Start with the **smoke test** (step 2): it checks the scripts without a GPU. If it has not passed by minute 15, put your hand up.

The full experiment is **one GPU job**. It scores the prompts, trains the model and runs the general-knowledge check, and it can take an hour or more. Submit it by minute 60, even with a first version of your prompt.

GPUs are reserved until about 20:00, so jobs can finish after the session.

Your data and your prompt

```
# 1. download an existing labelled dataset and split it (README step 5)
python3 from_huggingface.py --dataset stanfordnlp/sst2 --out sst2.jsonl
python3 make_splits.py --input sst2.jsonl --fields sentence \
    --label-field label --out data/my-task

# 2. lab2lib/prompting.py: the two settings you change (step 6)
TASK_FIELDS = ("sentence",)
TASK_SYSTEM = ("Is the review positive or negative? "
               "Use exactly one of: positive, negative. "
               "End with 'Label: <label>'.")
```

Three splits of your data

- **train**: what the model learns from (1,000–5,000)
- **dev**: for choosing the prompt and settings
- **test**: only for the final scores

You do **not** create a dataset. Use an existing labelled one: SST-2 (above), another dataset listed in the README, or your own file.

Keep the last sentence of the instruction: the scorer looks for `Label: <label>`. Step 7 checks the data and shows your prompts without a GPU.

The experiment

		README step	
Phase	**What happens**	**prepare**	**run → read**
1. Prompting	four prompts: base and instruct model, each zero-shot and with 5 examples; the best on dev is *the* baseline	6	8 → 9
2. Knowledge, before	the instruct model answers 50 general-knowledge questions	–	8 → 9
3. Fine-tuning	LoRA trains a small add-on to the instruct model on the train set; the loss plot shows training and dev loss	5, 7	8 → 9
4. Testing	the best prompt and the fine-tuned model are scored on the same test set	5	8 → 9
5. Knowledge, after	the fine-tuned model answers the same 50 questions; the change shows any damage	–	8 → 9
Conclusion	does fine-tuning beat the best prompt, and at what cost?		10

A **base model** has only learned to continue text; an **instruct model** has also learned to follow instructions. Steps refer to the README. Phases 1–5 all run in the one GPU job of step 8, in this order; its log shows them as stages `1/6` to `5/6`.

Read the outputs before you trust a score: a score of exactly 0 or 100 usually means the scorer is wrong, not the model (`show_examples.py`).

If you have time: tune the training

Setting	Start	What it controls
learning rate	0.0002	how large each training step is
epochs	2	how many times the model sees the training data
LoRA rank	16	the size of the adapter

Change *one* setting at a time, give each run its own name, and keep a small table of the runs. The README lists other extensions.

Lower training loss is not a result

The **loss** measures how wrong the model's answers are. If the training loss falls while the dev loss rises, the model is **memorising** the training examples.

Then try fewer epochs or a lower learning rate. Choose settings on the dev loss, never on the test score.

Checklist for today

[ ] smoke test passed (step 2)
[ ] example job finished on `gputest` (step 4)
[ ] dataset downloaded and split (step 5)
[ ] prompt adapted (step 6)
[ ] `check_data.py` clean, prompt read in a test run (step 7)
[ ] full job submitted to `gpumedium` (step 8)
[ ] table printed with `summarize_results.py`
[ ] ten raw outputs read with `show_examples.py`
[ ] results folder and config kept for the report (step 10)

Good practice on Roihu

- follow a job with `squeue --me` and `tail -f logs/lab2-lora-<jobid>.out`
- do not run models on the login node
- close GPU Jupyter sessions you are not using

Stuck for more than ten minutes? Put your hand up. The README has a troubleshooting table.